

PRÁCTICA 4

TEMPORIZADORES HARDWARE DEL ESP32-S3

Juan Pedro Bandera Rubio (jpbandera@uma.es)

El ESP32-S3 incluye temporizadores de propósito general (*General Purpose Timers* o *GPTimers*) que puedes utilizar para crear la base de tiempos de tus programas. El siguiente enlace proporciona información sobre las características y empleo de estos GPTimers, a través del driver GPTimer de Espressif: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/gptimer.html>

La primera cuestión que puede que te venga a la mente es: ¿para qué enredarme con los timers del ESP32-S3 cuando los timers FreeRTOS, que ya sé usar, funcionan bien?

La respuesta es: si no te importa que el jitter del temporizador sea algo peor, y no tienes problemas por la ejecución de muchas tareas, los timers FreeRTOS te van a ir estupendamente.

Brevemente, las ventajas que ofrece el uso de los GPTimer son las siguientes:

- Menos jitter (período más estable).
- Se emplean los recursos HW del micro, cargando menos el microcontrolador.
- La ejecución de una ISR es más eficiente que la ejecución de una tarea.

Puedes usar el código de ejemplo *ESP32_S3_espidf_TimerESP* para ver cómo se gestionan los GPTimer del ESP32-S3. En este código, fíjate que aparece una cosa nueva: una librería en el subdirectorio *lib*, llamaba *ub_esp32_s3_timerhw*.

LIBRERÍAS DE USUARIO EN PLATFORMIO

Crear y enlazar una librería a tu programa en PlatformIO es realmente fácil. Sólo tienes que crear una carpeta en el subdirectorio *lib* de tu proyecto, llamándola con el nombre que quieras, por ejemplo *mi_libreria*. Dentro de esa carpeta, creas un archivo de cabecera (*milibreria.h*) y otro de código (*milibreria.cpp*)... y ya está. Sólo tienes que rellenar esos archivos, incluir el archivo de cabecera en tu programa principal, y al compilar PlatformIO los gestionará automáticamente.

ARCHIVOS DE CABECERA DE USUARIO EN PLATFORMIO

Si sólo necesitas crear un archivo de cabecera (.h), se hace lo mismo que para las librerías, pero en este caso el archivo de cabecera lo creas directamente en el subdirectorio *include* de tu proyecto.

NOTA: Por si no te acuerdas, en tu proyecto en cada subcarpeta hay un README que cuenta exactamente lo mismo que acabas de leer, con más detalles de hecho.

LIBRERÍA UB_ESP32_S3_TIMERHW

Es la librería que hemos creado en la asignatura para facilitarte el control de los temporizadores del ESP32-S3 (y para animarte a usarlos). Básicamente la librería te ofrece estas funciones:

1.- Creación de un timer con una cierta configuración:

Esta es la función que inicializa y configura la base de tiempos del temporizador.

```
esp_err_t      ub_esp32_s3_timerhw_create(gptimer_clock_source_t      clk_src,
                                           gptimer_count_direction_t direction, uint32_t resolution_hz,
                                           gptimer_handle_t *timer_handle);
```

Los parámetros de entrada de la función son:

- `gptimer_clock_source_t clk_src` : **Fuente de reloj del temporizador.** Con la opción `GPTIMER_CLK_SRC_DEFAULT` seleccionas el reloj por defecto, que es el APB (Advanced Peripheral Bus Clock). A su vez, el APB depende del reloj de CPU. Si has dejado el reloj de CPU por defecto tras el arranque, que va a la mitad del cristal interno ($40\text{MHz} / 2 = 20\text{MHz}$), el APB va a esa misma velocidad (20MHz).
- `gptimer_count_direction_t direction` : **Dirección de cuenta.** Puedes contar hacia delante indicando aquí `GPTIMER_COUNT_UP` o hacia atrás con `GPTIMER_COUNT_DOWN`.
- `uint32_t resolution_hz` : **Resolución, en hercios (Hz).** Este parámetro fija el período del GPTimer. Indica, en hercios, cuánto tiempo tarda un *tick* de este temporizador, donde *tick* = *período de cuenta* (a partir de ahora usaremos el término *tick*, tal y como se usa en la documentación del ESP32). Así, si escribes aquí 1000000, eso es un millón de hercios (1 MHz, vaya), con lo que cada *tick* de tu GPTimer tendrá un período de $1/1000000 = 1\mu\text{s}$. O dicho de otro modo, tu temporizador tiene una base de tiempos de $1\mu\text{s}$ por *tick*.
- `gptimer_handle_t *timer_handle` : **Manejador del timer.** Tendrás que haber creado un manejador en tu programa antes de llamar a esta función. Aquí pasas ese manejador como parámetro. Consulta el código de ejemplo si tienes dudas con esto.

2.- Configuración de la alarma del temporizador:

Esta función permite definir cuándo va a saltar la interrupción del temporizador, y qué rutina ISR se ejecutará cuando eso ocurra.

```
esp_err_t ub_esp32_s3_timerhw_set_alarm(gptimer_handle_t timer_handle,
    uint64_t reload_count, uint64_t alarm_tick_count, bool auto_reload,
    gptimer_alarm_cb_t callback, void *user_data);
```

Los parámetros de entrada de la función son:

- **gptimer_handle_t timer_handle** : **Manejador del timer.**
- **uint64_t reload_count** : **Valor de recarga.** Si se configura el temporizador como de auto-recarga, este es el valor que se carga en el temporizador después de que haya saltado la alarma.
- **uint64_t alarm_tick_count** : **Valor en que salta la alarma (en ticks).** Cuando el valor de cuenta llega a este número de *ticks*, salta la alarma. Por tanto, si tienes un temporizador con una base de tiempos de 1µs por tick, y tu valor de recarga es 0, para que la alarma salte cada 2 segundos tendrías que poner aquí 2000000.
- **bool auto_reload** : **Auto-recarga.** Indicas si quieres que el temporizador cuente sólo una vez hasta que salte la alarma, o si quieres que se auto-recargue para generar alarmas periódicamente.
- **gptimer_alarm_cb_t callback** : **Rutina ISR (callback).** Rutina ISR que quieres que se ejecute cuando salte la alarma. Tendrás que definirla en tu programa principal, y recuerda que tienes que indicar que esa función se grabe en la RAM interna, con una cabecera que empiece así: **static bool IRAM_ATTR**
- **void *user_data** : **Datos de usuario para la rutina de callback.** Al igual que ocurría con las interrupciones de GPIO, es útil emplear una cola FreeRTOS para pasar datos a/desde la rutina ISR a tu programa. Aquí puedes pasar el manejador de esa cola. Consulta el código de ejemplo para ver cómo se hace esto. Si prefieres pasar datos de otra manera, también es posible.

3.- Funciones para habilitar y deshabilitar el temporizador, iniciar y parar la cuenta, y borrar el temporizador.

El resto de funciones en `ub_esp32_s3_timerhw` no requieren mucha explicación.

```
// enable the timer
    esp_err_t ub_esp32_s3_timerhw_enable(gptimer_handle_t timer_handle);
    // start the timer, the timer will start counting after this function
is called
    esp_err_t ub_esp32_s3_timerhw_start(gptimer_handle_t timer_handle);
// stop the timer, the timer will stop counting after this function is called
esp_err_t ub_esp32_s3_timerhw_stop(gptimer_handle_t timer_handle);    //
disable the timer. Disabled timers can be reconfigured
    esp_err_t ub_esp32_s3_timerhw_disable(gptimer_handle_t timer_handle);
    // delete the timer and free resources
```

```
esp_err_t ub_esp32_s3_timerhw_delete(gptimer_handle_t timer_handle);
```

Es importante, sin embargo, que recuerdes lo siguiente:

- No es necesario habilitar el temporizador al principio, ya que la función `ub_esp32_s3_timerhw_set_alarm` lo habilita internamente. Esto es así porque sólo pueden configurarse las alarmas de un temporizador habilitado.
- Sin embargo, si quieres reconfigurar la base de tiempos del temporizador, es necesario deshabilitarlo antes.
- Antes de cerrar el programa borra el manejador del temporizador para liberar recursos.

Si tienes dudas sobre las funciones que se usan en esta librería, recuerda que seleccionando el nombre de una función y pulsando F12 vas a su declaración, y allí te dan bastante información sobre qué es y cómo funciona cada cosa.